WEEK 8: DATABASE RELATIONSHIP ANALYSIS USING JOINS

1. Aim

To analyze the relationships between tables in the ArtHub database using SQL joins and generate meaningful reports about customers, artworks, orders, and payments.

2. Introduction

The ArtHub database stores information about customers, artworks, orders, order details, and payments in separate tables. These tables are related through common attributes such as CUSTOMER_ID, ORDER_ID, and ARTWORK_ID.

SQL joins are used to retrieve related information from multiple tables. They help identify customer purchases, display order details, analyze payment status, and prepare business reports.

3. Implement INNER JOIN

Concept:

An INNER JOIN returns only the records that have matching values in both joined tables.

Task 1: Display customer details along with their orders.

The Customer and Orders tables were joined using CUSTOMER_ID. The result displays customer information along with their order ID, order date, total amount, and order status.

```
SQL> SELECT Customer.Customer_ID, Customer.Customer_Name, Orders.Order_ID, Orders.Order_Date, Orders.Total_Amount, Orders.Order_Status FROM Customer INNER J
OIN Orders ON Customer.Customer_ID = Orders.Customer_ID;

CUSTOMER_ID CUSTOMER_NAME                  ORDER_ID   ORDER_DATE
----------- ------------------------------ ---------- ------------------
TOTAL_AMOUNT ORDER_STATUS
------------ --------------------
          1 Dhoni                          O1         20-SEP-26
        5000 Delivered

            2 Virat                        O2         21-SEP-26
        5200 Shipped

            3 Ashwin                       O3         21-SEP-26
        1800 Delivered


CUSTOMER_ID CUSTOMER_NAME                  ORDER_ID   ORDER_DATE
----------- ------------------------------ ---------- ------------------
TOTAL_AMOUNT ORDER_STATUS
------------ --------------------
          4 SanjuSamson                    O4         22-SEP-26
        2200 Processing

            5 Shreyas Iyer                 O5         22-SEP-26
        3500 Shipped

            6 Hardhik Pandya               O6         23-SEP-26
        4000 Delivered


CUSTOMER_ID CUSTOMER_NAME                  ORDER_ID   ORDER_DATE
----------- ------------------------------ ---------- ------------------
TOTAL_AMOUNT ORDER_STATUS
------------ --------------------
          7 Ruturaj Gaikwad                O7         24-SEP-26
        2800 Processing


7 rows selected.
```

Task 2: Display order details with payment information.

The Orders and Payment tables were joined using ORDER_ID. The result displays order details along with payment mode, payment status, and transaction amount.

```
SQL> SELECT Orders.Order_ID, Orders.Order_Date, Orders.Total_Amount, Payment.Payment_Mode, Payment.Payment_Status, Payment.Transaction_Amount FROM Orders IN
NER JOIN Payment ON Orders.Order_ID = Payment.Order_ID;

ORDER_ID   ORDER_DATE         TOTAL_AMOUNT PAYMENT_MODE
---------- ------------------ ------------ --------------------
PAYMENT_STATUS       TRANSACTION_AMOUNT
-------------------- ------------------
O1         20-SEP-26                5000 UPI
Completed                       5000

O2         21-SEP-26                5200 Card
Completed                       5200

O3         21-SEP-26                1800 UPI
Completed                       1800


ORDER_ID   ORDER_DATE         TOTAL_AMOUNT PAYMENT_MODE
---------- ------------------ ------------ --------------------
PAYMENT_STATUS       TRANSACTION_AMOUNT
-------------------- ------------------
O4         22-SEP-26                2200 Net Banking
Pending                         2200

O6         23-SEP-26                4000 UPI
Completed                       4000

O5         22-SEP-26                3500 Card
Completed                       3500


ORDER_ID   ORDER_DATE         TOTAL_AMOUNT PAYMENT_MODE
---------- ------------------ ------------ --------------------
PAYMENT_STATUS       TRANSACTION_AMOUNT
-------------------- ------------------
O7         24-SEP-26                2800 Card
Pending                         2800


7 rows selected.
```

Task 3: Retrieve products purchased by customers.

The Customer, Orders, OrderDetails, and ArtProduct tables were joined using their related IDs. The result displays customer names, order IDs, artwork details, and quantities purchased.

```
SQL> SELECT Customer.Customer_Name, Orders.Order_ID, ArtProduct.Artwork_ID, ArtProduct.Artwork_Name, OrderDetails.Quantity FROM Customer INNER JOIN Orders O
N Customer.Customer_ID = Orders.Customer_ID INNER JOIN OrderDetails ON Orders.Order_ID = OrderDetails.Order_ID INNER JOIN ArtProduct ON OrderDetails.Artwork
_ID = ArtProduct.Artwork_ID;

CUSTOMER_NAME                ORDER_ID   ARTWORK_ID
---------------------------- ---------- ----------
ARTWORK_NAME
--------------------------------------------------------------------------------
  QUANTITY
----------
Dhoni                        O1         101
Sunset Over Hills
         1

Virat                        O2         102
Colors of Nature
         1

CUSTOMER_NAME                ORDER_ID   ARTWORK_ID
---------------------------- ---------- ----------
ARTWORK_NAME
--------------------------------------------------------------------------------
  QUANTITY
----------

Ashwin                       O3         103
Pencil Portrait
         1

SanjuSamson                  O4         104
Charcoal Dreams

CUSTOMER_NAME                ORDER_ID   ARTWORK_ID
---------------------------- ---------- ----------
ARTWORK_NAME
--------------------------------------------------------------------------------
  QUANTITY
----------
         1
```

4. Implement LEFT JOIN

Concept:

A LEFT JOIN returns all records from the left table and matching records from the right table. If no matching record exists, the right-side columns contain NULL values.

Task 1: Display all customers, including customers who have not placed orders.

The Customer table was joined with the Orders table using LEFT JOIN. This displays all customers, including those without any orders.

```
SQL> SELECT Customer.Customer_ID, Customer.Customer_Name, Orders.Order_ID, Orders.Total_Amount FROM Customer LEFT JOIN Orders ON Customer.Customer_ID = Orde
rs.Customer_ID;

CUSTOMER_ID CUSTOMER_NAME                  ORDER_ID   TOTAL_AMOUNT
----------- ------------------------------ ---------- ------------
          1 Dhoni                          O1               5000
          2 Virat                          O2               5200
          3 Ashwin                         O3               1800
          4 SanjuSamson                    O4               2200
          5 Shreyas Iyer                   O5               3500
          6 Hardhik Pandya                 O6               4000
          7 Ruturaj Gaikwad                O7               2800
          8 Rohit Sharma
         10 Jadeja
          9 Raina

10 rows selected.
```

Task 2: Find customers without any purchases.

A LEFT JOIN was used between Customer and Orders, followed by a condition to identify customers whose order ID is NULL. The result identifies customers who have not placed orders.

```
SQL> SELECT Customer.Customer_ID, Customer.Customer_Name FROM Customer LEFT JOIN Orders ON Customer.Customer_ID = Orders.Customer_ID WHERE Orders.Order_ID I
S NULL;

CUSTOMER_ID CUSTOMER_NAME
----------- ------------------------------
          8 Rohit Sharma
         10 Jadeja
          9 Raina
```

Task 3: Display all artworks, including artworks with no sales.

The ArtProduct table was joined with OrderDetails using LEFT JOIN. This displays all artworks and their corresponding order details, where available.

```
SQL> SELECT ArtProduct.Artwork_ID, ArtProduct.Artwork_Name, OrderDetails.Order_ID, OrderDetails.Quantity FROM ArtProduct LEFT JOIN OrderDetails ON ArtProduc
t.Artwork_ID = OrderDetails.Artwork_ID;

ARTWORK_ID
----------
ARTWORK_NAME
--------------------------------------------------------------------------------
ORDER_ID     QUANTITY
---------- ----------
101
Sunset Over Hills
O1                  1

102
Colors of Nature
O2                  1

ARTWORK_ID
----------
ARTWORK_NAME
--------------------------------------------------------------------------------
ORDER_ID     QUANTITY
---------- ----------

103
Pencil Portrait
O3                  1

104
Charcoal Dreams

ARTWORK_ID
----------
ARTWORK_NAME
--------------------------------------------------------------------------------
ORDER_ID     QUANTITY
---------- ----------
O4                  1

105
```

```
ARTWORK_ID
----------
ARTWORK_NAME
--------------------------------------------------------------------------------
ORDER_ID     QUANTITY
---------- ----------
Future City
O6                  1

109
Abstract Dreams


ARTWORK_ID
----------
ARTWORK_NAME
--------------------------------------------------------------------------------
ORDER_ID     QUANTITY
---------- ----------
110
Handmade Craft


108
The Thinker Sculpture

ARTWORK_ID
----------
ARTWORK_NAME
--------------------------------------------------------------------------------
ORDER_ID     QUANTITY
---------- ----------

107
Morning Light


10 rows selected.
```

5. Implement RIGHT JOIN

Concept:

A RIGHT JOIN returns all records from the right table and matching records from the left table. If no match exists, the left-side columns contain NULL values.

Task 1: Display all orders with customer information.

The Customer and Orders tables were joined using RIGHT JOIN. The result displays every order with the corresponding customer information, where available.

```
SQL> SELECT Orders.Order_ID, Orders.Order_Date, Orders.Total_Amount, Orders.Order_Status, Customer.Customer_ID, Customer.Customer_Name FROM Customer RIGHT J
OIN Orders ON Customer.Customer_ID = Orders.Customer_ID;

ORDER_ID   ORDER_DATE         TOTAL_AMOUNT ORDER_STATUS         CUSTOMER_ID
---------- ------------------ ------------ -------------------- -----------
CUSTOMER_NAME
--------------------------------
O1         20-SEP-26                  5000 Delivered                      1
Dhoni

O2         21-SEP-26                  5200 Shipped                        2
Virat

O3         21-SEP-26                  1800 Delivered                      3
Ashwin


ORDER_ID   ORDER_DATE         TOTAL_AMOUNT ORDER_STATUS         CUSTOMER_ID
---------- ------------------ ------------ -------------------- -----------
CUSTOMER_NAME
--------------------------------
O4         22-SEP-26                  2200 Processing                     4
SanjuSamson

O5         22-SEP-26                  3500 Shipped                        5
Shreyas Iyer

O6         23-SEP-26                  4000 Delivered                      6
Hardhik Pandya


ORDER_ID   ORDER_DATE         TOTAL_AMOUNT ORDER_STATUS         CUSTOMER_ID
---------- ------------------ ------------ -------------------- -----------
CUSTOMER_NAME
--------------------------------
O7         24-SEP-26                  2800 Processing                     7
Ruturaj Gaikwad


7 rows selected.
```

Task 2: Find orders where customer details are missing.

A RIGHT JOIN was used between Customer and Orders, with a condition to find records where the customer details are NULL. The result displayed “no rows selected,” indicating that every order currently has a matching customer record.

```
SQL> SELECT Orders.Order_ID, Orders.Customer_ID, Customer.Customer_Name FROM Customer RIGHT JOIN Orders ON Customer.Customer_ID = Orders.Customer_ID WHERE C
ustomer.Customer_ID IS NULL;

no rows selected
```

Task 3: Display all payment records with order details.

The Orders and Payment tables were joined using RIGHT JOIN. The result displays every payment record along with the corresponding order information, where available.

```
SQL> SELECT Payment.Payment_ID, Payment.Order_ID, Payment.Payment_Mode, Payment.Payment_Status, Payment.Transaction_Amount, Orders.Order_Date, Orders.Total_
Amount FROM Orders RIGHT JOIN Payment ON Orders.Order_ID = Payment.Order_ID;

PAYMENT_ID ORDER_ID   PAYMENT_MODE         PAYMENT_STATUS
---------- ---------- -------------------- --------------------
TRANSACTION_AMOUNT ORDER_DATE         TOTAL_AMOUNT
------------------ ------------------ ------------
P1         O1         UPI                  Completed
              5000 20-SEP-26                  5000

P2         O2         Card                 Completed
              5200 21-SEP-26                  5200

P3         O3         UPI                  Completed
              1800 21-SEP-26                  1800


PAYMENT_ID ORDER_ID   PAYMENT_MODE         PAYMENT_STATUS
---------- ---------- -------------------- --------------------
TRANSACTION_AMOUNT ORDER_DATE         TOTAL_AMOUNT
------------------ ------------------ ------------
P4         O4         Net Banking          Pending
              2200 22-SEP-26                  2200

P5         O5         Card                 Completed
              3500 22-SEP-26                  3500

P6         O6         UPI                  Completed
              4000 23-SEP-26                  4000


PAYMENT_ID ORDER_ID   PAYMENT_MODE         PAYMENT_STATUS
---------- ---------- -------------------- --------------------
TRANSACTION_AMOUNT ORDER_DATE         TOTAL_AMOUNT
------------------ ------------------ ------------
P7         O7         Card                 Pending
              2800 24-SEP-26                  2800


7 rows selected.
```

6. Retrieve Complete Order Details

A complete order report was generated by joining the Customer, Orders, OrderDetails, ArtProduct, and Payment tables.

The report contains the following information:

- Customer name
- Artwork name
- Quantity purchased
- Order date
- Total order amount
- Payment status

This report combines customer, artwork, order, and payment information into a single result, making it easier to understand each purchase.

```
SQL> SELECT Customer.Customer_Name, ArtProduct.Artwork_Name, OrderDetails.Quantity, Orders.Order_Date, Orders.Total_Amount, Payment.Payment_Status FROM Cust
omer INNER JOIN Orders ON Customer.Customer_ID = Orders.Customer_ID INNER JOIN OrderDetails ON Orders.Order_ID = OrderDetails.Order_ID INNER JOIN ArtProduct
 ON OrderDetails.Artwork_ID = ArtProduct.Artwork_ID INNER JOIN Payment ON Orders.Order_ID = Payment.Order_ID;

CUSTOMER_NAME
------------------------------
ARTWORK_NAME
--------------------------------------------------------------------------------
  QUANTITY ORDER_DATE         TOTAL_AMOUNT PAYMENT_STATUS
---------- ------------------ ------------ --------------------
Dhoni
Sunset Over Hills
         1 20-SEP-26                  5000 Completed

Virat
Colors of Nature
         1 21-SEP-26                  5200 Completed

CUSTOMER_NAME
------------------------------
ARTWORK_NAME
--------------------------------------------------------------------------------
  QUANTITY ORDER_DATE         TOTAL_AMOUNT PAYMENT_STATUS
---------- ------------------ ------------ --------------------

Ashwin
Pencil Portrait
         1 21-SEP-26                  1800 Completed

SanjuSamson
Charcoal Dreams

CUSTOMER_NAME
------------------------------
ARTWORK_NAME
--------------------------------------------------------------------------------
  QUANTITY ORDER_DATE         TOTAL_AMOUNT PAYMENT_STATUS
---------- ------------------ ------------ --------------------
```

```
CUSTOMER_NAME
--------------------------------
ARTWORK_NAME
--------------------------------------------------------------------------------
  QUANTITY ORDER_DATE           TOTAL_AMOUNT PAYMENT_STATUS
---------- -------------------- ------------ --------------------

Ashwin
Pencil Portrait
         1 21-SEP-26                    1800 Completed

SanjuSamson
Charcoal Dreams

CUSTOMER_NAME
--------------------------------
ARTWORK_NAME
--------------------------------------------------------------------------------
  QUANTITY ORDER_DATE           TOTAL_AMOUNT PAYMENT_STATUS
---------- -------------------- ------------ --------------------
         1 22-SEP-26                    2200 Pending

Shreyas Iyer
Digital Galaxy
         1 22-SEP-26                    3500 Completed

Hardhik Pandya

CUSTOMER_NAME
--------------------------------
ARTWORK_NAME
--------------------------------------------------------------------------------
  QUANTITY ORDER_DATE           TOTAL_AMOUNT PAYMENT_STATUS
---------- -------------------- ------------ --------------------
Future City
         1 23-SEP-26                    4000 Completed


6 rows selected.
```

7. Display Customer Purchase History

Task 1: Display all products purchased by customers.

The Customer, Orders, OrderDetails, and ArtProduct tables were joined to display the artworks purchased by customers, together with the corresponding order IDs and quantities.

```
SQL> SELECT Customer.Customer_Name, ArtProduct.Artwork_Name, OrderDetails.Quantity, Orders.Order_ID FROM Customer INNER JOIN Orders ON Customer.Customer_ID
= Orders.Customer_ID INNER JOIN OrderDetails ON Orders.Order_ID = OrderDetails.Order_ID INNER JOIN ArtProduct ON OrderDetails.Artwork_ID = ArtProduct.Artwor
k_ID;

CUSTOMER_NAME
------------------------------
ARTWORK_NAME
--------------------------------------------------------------------------------
  QUANTITY ORDER_ID
---------- ----------
Dhoni
Sunset Over Hills
         1 O1

Virat
Colors of Nature
         1 O2

CUSTOMER_NAME
------------------------------
ARTWORK_NAME
--------------------------------------------------------------------------------
  QUANTITY ORDER_ID
---------- ----------

Ashwin
Pencil Portrait
         1 O3

SanjuSamson
Charcoal Dreams

CUSTOMER_NAME
------------------------------
ARTWORK_NAME
--------------------------------------------------------------------------------
  QUANTITY ORDER_ID
---------- ----------
```

```
CUSTOMER_NAME
------------------------------
ARTWORK_NAME
--------------------------------------------------------------------------------
  QUANTITY ORDER_ID
---------- ----------

Ashwin
Pencil Portrait
         1 O3

SanjuSamson
Charcoal Dreams

CUSTOMER_NAME
------------------------------
ARTWORK_NAME
--------------------------------------------------------------------------------
  QUANTITY ORDER_ID
---------- ----------
         1 O4

Shreyas Iyer
Digital Galaxy
         1 O5

Hardhik Pandya

CUSTOMER_NAME
------------------------------
ARTWORK_NAME
--------------------------------------------------------------------------------
  QUANTITY ORDER_ID
---------- ----------
Future City
         1 O6


6 rows selected.
```

Task 2: Display the total amount spent by each customer.

A LEFT JOIN and the SUM aggregate function were used to calculate the total order amount for each customer. Customers without orders were also included, with their spending displayed as zero.

```
SQL> SELECT Customer.Customer_ID, Customer.Customer_Name, NVL(SUM(Orders.Total_Amount), 0) AS Total_Amount_Spent FROM Customer LEFT JOIN Orders ON Customer.
Customer_ID = Orders.Customer_ID GROUP BY Customer.Customer_ID, Customer.Customer_Name ORDER BY Customer.Customer_ID;

CUSTOMER_ID CUSTOMER_NAME                 TOTAL_AMOUNT_SPENT
----------- ------------------------------ ------------------
          1 Dhoni                                        5000
          2 Virat                                        5200
          3 Ashwin                                       1800
          4 SanjuSamson                                  2200
          5 Shreyas Iyer                                 3500
          6 Hardhik Pandya                               4000
          7 Ruturaj Gaikwad                              2800
          8 Rohit Sharma                                    0
          9 Raina                                           0
         10 Jadeja                                          0

10 rows selected.
```

Task 3: Display the number of orders placed by each customer.

A LEFT JOIN and the COUNT aggregate function were used to count the orders placed by each customer. Customers without orders were included in the result.

```
SQL> SELECT Customer.Customer_Name, Orders.Order_ID, Orders.Order_Date, Orders.Total_Amount FROM Customer INNER JOIN Orders ON Customer.Customer_ID = Orders
.Customer_ID WHERE Orders.Order_Date = (SELECT MAX(O2.Order_Date) FROM Orders O2 WHERE O2.Customer_ID = Customer.Customer_ID);

CUSTOMER_NAME                 ORDER_ID   ORDER_DATE        TOTAL_AMOUNT
----------------------------- ---------- ----------------- ------------
Dhoni                         01         20-SEP-26                5000
Virat                         02         21-SEP-26                5200
Ashwin                        03         21-SEP-26                1800
SanjuSamson                   04         22-SEP-26                2200
Shreyas Iyer                  05         22-SEP-26                3500
Hardhik Pandya                06         23-SEP-26                4000
Ruturaj Gaikwad               07         24-SEP-26                2800

7 rows selected.
```

Task 4: Display the latest purchase details of customers.

The maximum order date was identified for each customer using a subquery. The result displays the latest order ID, order date, and total amount for each customer who has placed an order.

```
SQL> SELECT Customer.Customer_Name, Orders.Order_ID, Orders.Order_Date, Orders.Total_Amount FROM Customer INNER JOIN Orders ON Customer.Customer_ID = Orders
.Customer_ID WHERE Orders.Order_Date = (SELECT MAX(O2.Order_Date) FROM Orders O2 WHERE O2.Customer_ID = Customer.Customer_ID);

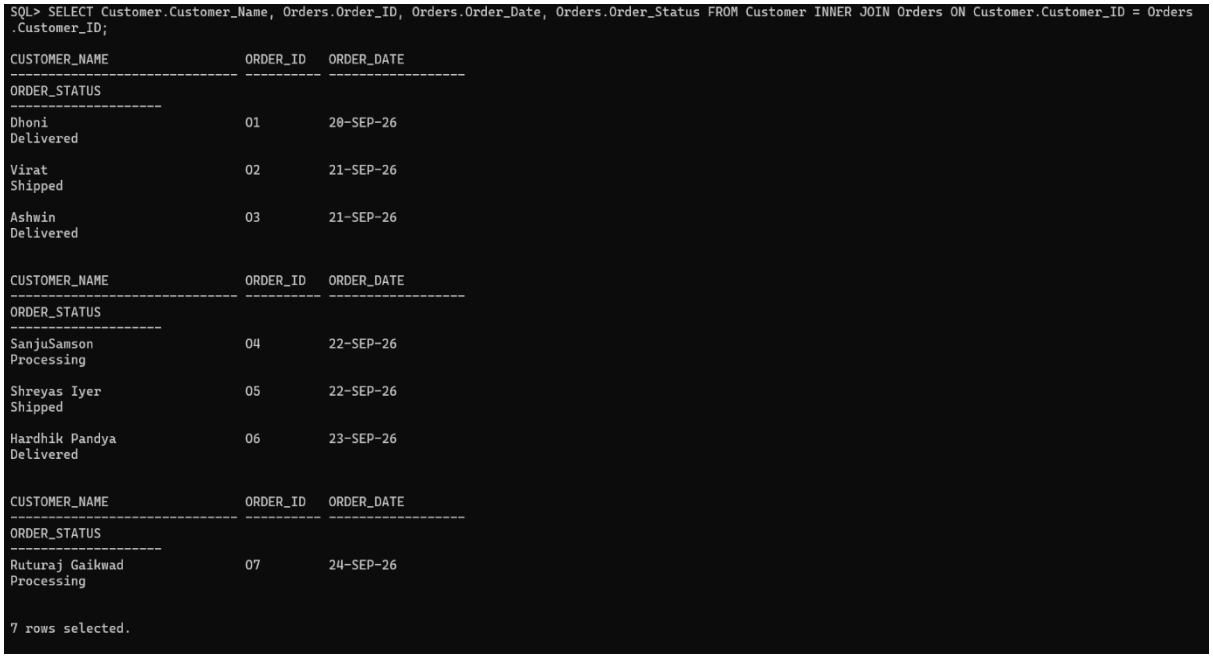
CUSTOMER_NAME                 ORDER_ID   ORDER_DATE        TOTAL_AMOUNT
----------------------------- ---------- ----------------- ------------
Dhoni                         01         20-SEP-26                5000
Virat                         02         21-SEP-26                5200
Ashwin                        03         21-SEP-26                1800
SanjuSamson                   04         22-SEP-26                2200
Shreyas Iyer                  05         22-SEP-26                3500
Hardhik Pandya                06         23-SEP-26                4000
Ruturaj Gaikwad               07         24-SEP-26                2800

7 rows selected.
```

8. Generate Multi-Table Business Reports

Report 1: Customer Order Report

The Customer and Orders tables were joined to display the customer name, order ID, order date, and order status. This report helps track customer orders and their current status.



```
SQL> SELECT Customer.Customer_Name, Orders.Order_ID, Orders.Order_Date, Orders.Order_Status FROM Customer INNER JOIN Orders ON Customer.Customer_ID = Orders
.Customer_ID;

CUSTOMER_NAME                  ORDER_ID   ORDER_DATE
------------------------------ ---------- ------------------
ORDER_STATUS
--------------------
Dhoni                          O1         20-SEP-26
Delivered

Virat                          O2         21-SEP-26
Shipped

Ashwin                         O3         21-SEP-26
Delivered


CUSTOMER_NAME                  ORDER_ID   ORDER_DATE
------------------------------ ---------- ------------------
ORDER_STATUS
--------------------
SanjuSamson                    O4         22-SEP-26
Processing

Shreyas Iyer                   O5         22-SEP-26
Shipped

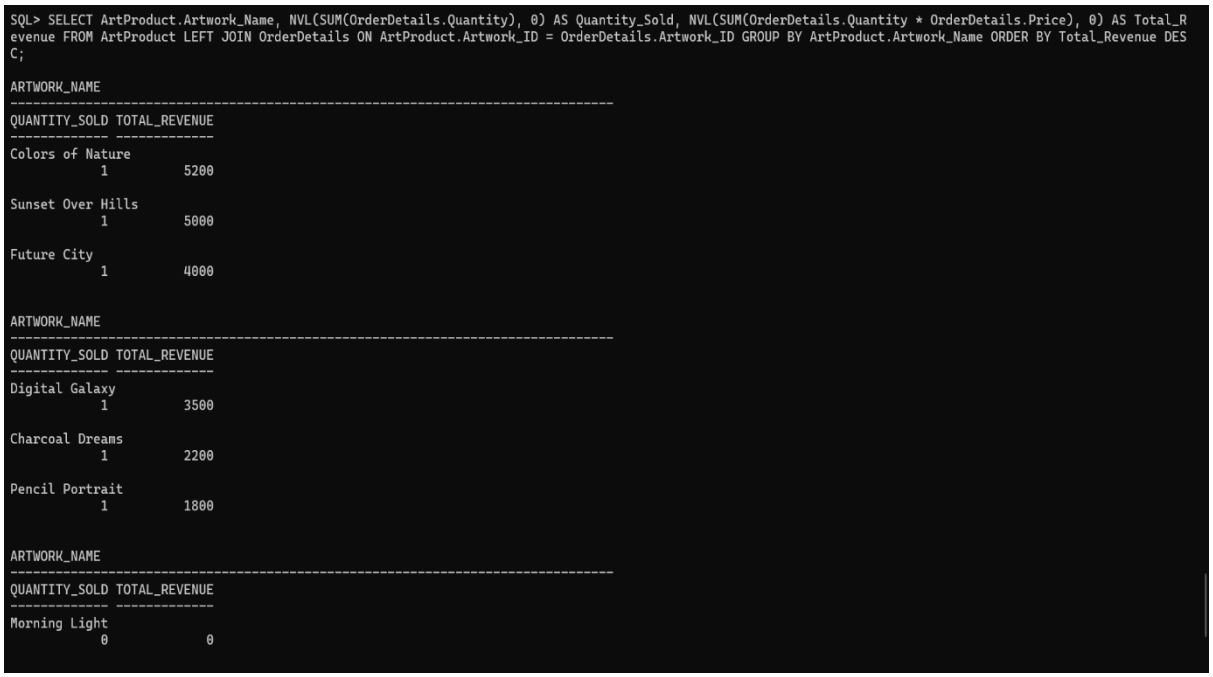
Hardhik Pandya                 O6         23-SEP-26
Delivered


CUSTOMER_NAME                  ORDER_ID   ORDER_DATE
------------------------------ ---------- ------------------
ORDER_STATUS
--------------------
Ruturaj Gaikwad                O7         24-SEP-26
Processing


7 rows selected.
```

Report 2: Sales Report

The ArtProduct and OrderDetails tables were joined to calculate the quantity sold and total revenue for each artwork. Aggregate functions were used to summarize the sales information.



```
SQL> SELECT ArtProduct.Artwork_Name, NVL(SUM(OrderDetails.Quantity), 0) AS Quantity_Sold, NVL(SUM(OrderDetails.Quantity * OrderDetails.Price), 0) AS Total_R
evenue FROM ArtProduct LEFT JOIN OrderDetails ON ArtProduct.Artwork_ID = OrderDetails.Artwork_ID GROUP BY ArtProduct.Artwork_Name ORDER BY Total_Revenue DES
C;

ARTWORK_NAME
--------------------------------------------------------------------------------
QUANTITY_SOLD TOTAL_REVENUE
------------- -------------
Colors of Nature
            1          5200

Sunset Over Hills
            1          5000

Future City
            1          4000


ARTWORK_NAME
--------------------------------------------------------------------------------
QUANTITY_SOLD TOTAL_REVENUE
------------- -------------
Digital Galaxy
            1          3500

Charcoal Dreams
            1          2200

Pencil Portrait
            1          1800


ARTWORK_NAME
--------------------------------------------------------------------------------
QUANTITY_SOLD TOTAL_REVENUE
------------- -------------
Morning Light
            0             0
```

```
ARTWORK_NAME
--------------------------------------------------------------------
QUANTITY_SOLD TOTAL_REVENUE
------------- -------------
Digital Galaxy
            1          3500

Charcoal Dreams
            1          2200

Pencil Portrait
            1          1800

ARTWORK_NAME
--------------------------------------------------------------------
QUANTITY_SOLD TOTAL_REVENUE
------------- -------------
Morning Light
            0             0

Abstract Dreams
            0             0

The Thinker Sculpture
            0             0

ARTWORK_NAME
--------------------------------------------------------------------
QUANTITY_SOLD TOTAL_REVENUE
------------- -------------
Handmade Craft
            0             0

10 rows selected.
```

Report 3: Payment Analysis Report

The Payment table was grouped by payment mode. The COUNT function was used to calculate the number of transactions, while a conditional SUM was used to count successful payments marked as Completed.

```
SQL> SELECT Payment_Mode, COUNT(*) AS Total_Transactions, SUM(CASE WHEN Payment_Status = 'Completed' THEN 1 ELSE 0 END) AS Successful_Payments FROM Payment
GROUP BY Payment_Mode;

PAYMENT_MODE         TOTAL_TRANSACTIONS SUCCESSFUL_PAYMENTS
-------------------- ------------------ -------------------
Net Banking                           1                   0
Card                                  3                   2
UPI                                   3                   3
```

Report 4: Customer Purchase Analysis

This report was divided into three analyses.

a. Top purchasing customers

Customers were ranked according to the total quantity of artworks purchased. This helps identify customers who purchased the greatest number of items.

```
SQL> SELECT Customer.Customer_ID, Customer.Customer_Name, SUM(OrderDetails.Quantity) AS Total_Items_Purchased FROM Customer INNER JOIN Orders ON Customer.Cu
stomer_ID = Orders.Customer_ID INNER JOIN OrderDetails ON Orders.Order_ID = OrderDetails.Order_ID GROUP BY Customer.Customer_ID, Customer.Customer_Name ORDE
R BY Total_Items_Purchased DESC;

CUSTOMER_ID CUSTOMER_NAME                 TOTAL_ITEMS_PURCHASED
----------- ------------------------------ ---------------------
          4 SanjuSamson                                      1
          1 Dhoni                                            1
          2 Virat                                            1
          3 Ashwin                                           1
          5 Shreyas Iyer                                     1
          6 Hardhik Pandya                                   1

6 rows selected.
```

b. Customers with maximum orders

The number of orders placed by each customer was calculated and compared with the highest order count. This identifies the customer or customers with the maximum number of orders.

```
SQL> SELECT Customer.Customer_ID, Customer.Customer_Name, COUNT(Orders.Order_ID) AS Total_Orders FROM Customer INNER JOIN Orders ON Customer.Customer_ID = O
rders.Customer_ID GROUP BY Customer.Customer_ID, Customer.Customer_Name HAVING COUNT(Orders.Order_ID) = (SELECT MAX(Order_Count) FROM (SELECT COUNT(*) AS Or
der_Count FROM Orders GROUP BY Customer_ID));

CUSTOMER_ID CUSTOMER_NAME                  TOTAL_ORDERS
----------- ------------------------------ ------------
          4 SanjuSamson                             1
          1 Dhoni                                   1
          6 Hardhik Pandya                          1
          3 Ashwin                                  1
          5 Shreyas Iyer                            1
          2 Virat                                   1
          7 Ruturaj Gaikwad                         1

7 rows selected.
```

c. Customers with highest spending

The total order amount was calculated for each customer. The customer or customers with the highest total spending were displayed.

```
SQL> SELECT Customer.Customer_ID, Customer.Customer_Name, SUM(Orders.Total_Amount) AS Total_Spending FROM Customer INNER JOIN Orders ON Customer.Customer_ID
 = Orders.Customer_ID GROUP BY Customer.Customer_ID, Customer.Customer_Name HAVING SUM(Orders.Total_Amount) = (SELECT MAX(Total_Spending) FROM (SELECT SUM(T
otal_Amount) AS Total_Spending FROM Orders GROUP BY Customer_ID));

CUSTOMER_ID CUSTOMER_NAME                  TOTAL_SPENDING
----------- ------------------------------ --------------
          2 Virat                                    5200
```

9. Result

The Week 8 database relationship analysis was carried out using INNER JOIN, LEFT JOIN, and RIGHT JOIN operations on the ArtHub database. Customer purchase history, complete order details, sales information, payment analysis, and customer purchase reports were generated by combining related tables.

The exercise demonstrated how SQL joins and aggregate functions can be used to retrieve meaningful information from a relational database and support business analysis.